

# 算法简介

---

什么是算法：算法是带有输入输出的、**有限的**、**确定的**、有效的过程

---

程序与算法的联系：程序是算法用某种程序设计语言的具体实现

程序与算法的区别：

- 在语言描述上，程序必须是用规定的程序设计语言来写，而算法很随意；
- 在执行时间上，算法所描述的步骤一定是有限的，而程序可以无限地执行下去。

数据结构——高效地组织计算的对象或者要处理的信息

---

## 经典排序算法

---

经验法评估复杂度的问题：1、依赖于运行环境2、依赖于语言/编程技能3、需要调试时间4、只能评估部分实例的效果

---

渐进上界 $O$

$\exists c, n_0 \in N^+$  , 使得对 $\forall n \geq n_0, T(n) \leq cf(n)$ , 则  $T(n) = O(f(n))$ .

渐进下界 $\Omega$

...

紧渐进界 $\Theta$

$\exists c_1, c_2, n_0 \in N^+$ , 使得对 $\forall n \geq n_0, c_1 f(n) \leq T(n) \leq c_2 f(n)$ , 则  $T(n) = \Theta(f(n))$

☆  $O$  的运算性质

$$O(f) + O(g) = O(\max\{f, g\})$$

$$O(f) + O(g) = O(f + g)$$

$$O(f) * O(g) = O(f * g)$$

如果  $g = O(f)$ , 则  $O(f) + O(g) = O(f)$

---

# NP完全性理论

---

易解问题：存在多项式时间算法（排序、查找、欧拉回路

难解问题：需要指数时间算法（TSP、Hanio、Hamilton回路

判定问题：要求回答true或者false

---

确定性算法与P问题

确定性算法：在求解问题的过程中每步只有一个确定的选择

P问题：对于判定问题，给定输入规模为n的实例，能够以 $O(n^k)$ 运行一个确定性算法，则该判定问题是P问题。

- 所有易解问题都是P类问题

---

## 非确定性算法与NP问题

非确定性算法：使用猜测并验证的方式解决问题（先根据输入实例产生一个结果，再使用确定性算法进行验证）

NP问题：对于判定问题，给定输入规模为n的实例，能够以 $O(n^k)$ 运行一个**非确定性算法**，则该判定问题是NP问题

- $P \subseteq NP$

---

## NP完全问题（NPC）和NP难问题

NPC问题一定是NP问题，NPH问题不一定是NP问题

- 一般而言，若判定问题属于NP完全问题，则相应的最优化问题属于NP难问题。

---

# 分治和递归

## 递归

- 整数划分问题（双变量）
- 汉诺塔问题（n个盘子借助B, 从A移到C）

递归的层次——调用递归函数的主函数为第0层，首次递归调用为第1层。

复杂度求解——迭代法+数学归纳法证明

---

## 分治法

基本思想——将一个规模为n的问题分解为k个规模较小的子问题，这些子问题互相独立且与原问题相同。递归解这些子问题，再将子问题合并得到原问题的解。

适用条件——1、小问题可解2、可以分解成若干相同的子问题（最优子结构性质）3、子问题之和可以得出原问题4、子问题相互独立

分治法的时间复杂度

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(\frac{n}{m}) + f(n) & n > 1 \end{cases}$$
$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(\frac{n}{m^j})$$

- 画出递归树
- **主定理**

对于分治法 $T = aT(\frac{n}{b}) + f(n)$

1. 若 $f(n) = O(n^{\log_b a - \epsilon})$ , 则 $T(n) = \Theta(n^{\log_b a})$ .
2. 若 $f(n) = \Theta(n^{\log_b a})$ , 则 $T(n) = \Theta(n^{\log_b a} \log n)$

3. 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$ , 并且对于充分大的 $n$ , 有 $af(\frac{n}{b}) \leq cf(n)$  ( $c < 1$  为常数), 则  
 $T(n) = \Theta(f(n))$

不能使用主定理时（相差 $\log n$ ）可以考虑递归树

## 动态规划

最优化问题：

- 解空间： $n$ 个输入
- 可行解：满足约束条件的解
- 最优解：使得目标函数取得极值的可行解

多阶段决策：每个阶段的决策仅仅依赖于前一阶段的状态

最优性原理：无论决策过程的初始状态和 初始决策是什么，其余的决策都必须相对于初始 决策所产生的当前状态，构成一个最优决策序列。（整体最优包含局部最优）

DP的基本思想——储存已解决问题的答案（自底向上

☆一般建立二元函数，其中迭代过程采用穷举法列出可能情况取极值

经典问题：矩阵连乘、最长公共子序列、完全背包问题

## 贪心算法

贪心：不从整体最优出发，仅选局部最优

贪心算法的条件：

- 最优性原理
- 贪心选择性质：指问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来得到。

贪心算法的证明

- 数学归纳法

经典问题：活动选择、装载问题、二元前缀码、最小生成树（Prim、Kruskal）、最短路（Dijkstra）

## 回溯与分支限界

回溯法与分支限界的区别：

|      | 回溯法  | 分支限界          |
|------|------|---------------|
| 搜索方式 | 深度优先 | 广度优先或优先级优先    |
| 搜索目标 | 所有解  | 最优解           |
| 搜索函数 | 剪枝函数 | 剪枝函数+限界、优先级函数 |

分支限界优先级队列：一次性产生所有儿子结点，其中不可行解与非最优解被抛弃，其余的解被加入队列中